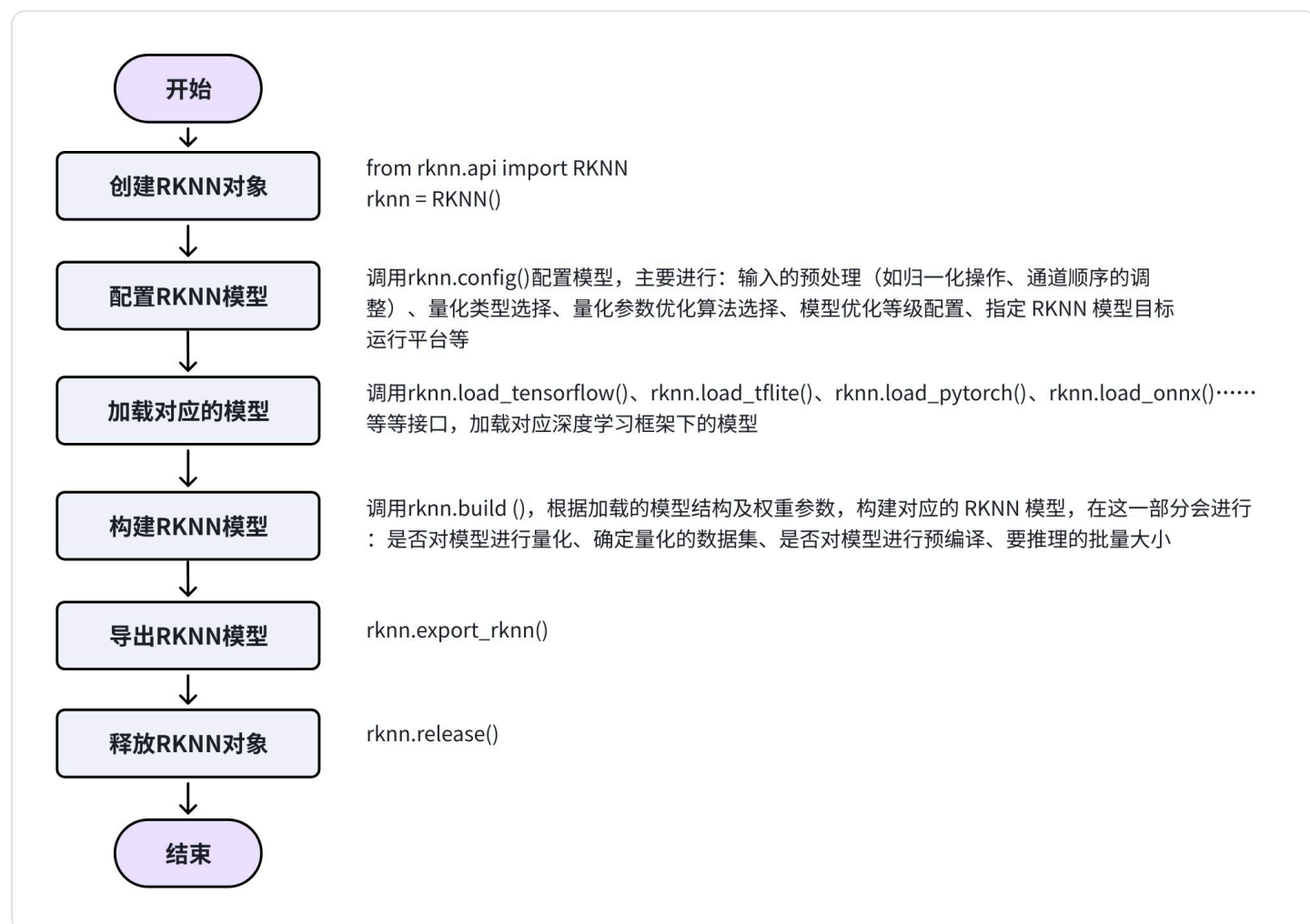


9.RKNN模型转换

1. 模型转换流程



目前RKNN-Toolkit2支持多个主流深度学习框架的模型转换，包括：

- Caffe（推荐版本为1.0）
- TensorFlow（推荐版本为1.12.0~2.8.0）
- TensorFlow Lite（推荐版本为Schema version = 3）
- ONNX（推荐版本为1.7.0~1.10.0）
- PyTorch（推荐版本为1.6.0~1.13.1）
- Darknet（推荐版本为Commit ID = 810d7f7）

2. RKNN-Toolkit2 API 介绍

2.1 RKNN初始化及对象释放

在使用RKNN-Toolkit2的所有API接口时，都需要先调用RKNN()方法初始化RKNN对象，不再使用该对象时通过调用该对象的release()方法进行释放。

初始化RKNN对象时，可以设置verbose和verbose_file参数，以打印详细的日志信息。其中verbose参数指定是否要打印详细日志信息；如果设置了verbose_file参数，且verbose参数值为True，日志信息还将写到该参数指定的文件中。

举例如下：

代码块

```
1 // 将详细的日志信息输出到终端，并写到 mobilenet_build.log 文件中
2 rknn = RKNN(verbose=True,verbose_file='./mobilenet_build.log')
3 rknn = RKNN(verbose=True) #将详细的日志信息输出到终端
4 ...
5 rknn.release()
```

2.2 模型转换配置

在构建RKNN模型之前，需要先对模型进行通道均值、量化图片RGB2BGR转换、量化类型等的配置，这些操作可以通过config接口进行配置。

参数	描述
mean_values	输入的均值。参数格式是一个列表，列表中包含一个或多个均值子列表，多输入模型对应多个子列表，每个子列表的长度与该输入的通道数一致，例如 [[128,128,128]]，表示一个输入的三个通道的值减去 128。如果 quant_img_RGB2BGR 设置 成 True，则优先做 RGB2BGR 转换，再做减均值。
std_values	输入的归一化值。参数格式是一个列表，列表中包含一个或多个归一化值子列表，多输入模型对应多个子列表，每个子列表的长度与该输入的通道数一致，例如 [[128,128,128]]，表示设置一个输入的三个通道的值减去均值后再除以128。默认值为None，表示所有的std值为1。
quant_img_RGB2BGR	表示在加载量化图像时是否需要先做RGB2BGR的操作。如果有多个输入，则用列表包含起来，如[True, True, False]。默认值为False。 该配置一般用在Caffe的模型上，Caffe模型训练时大多会先对数据集图像进行RGB2BGR转换，此时需将该配置设为True。 另外，该配置只对量化图像格式为jpg/png/bmp有效，npz格式读取时会忽略该配置，因此当 模型输入为BGR时，npz也需要为BGR格式。 该配置仅用于在量化阶段（build接口）读取量化图像或量化精度分析（accuracy_analysis接

	口），并不会保存在最终的RKNN模型中，因此如果模型的输入为BGR，则在调用 toolkit2 的inference或C-API的run函数之前，需要保证传入的图像数据也为BGR格式。
quantized_dtype	量化类型，目前支持的量化类型有w8a8、w4a16、w8a16、w4a8、w16a16i和w16a16i_dfp。默认值为w8a8。 • w8a8：权重为8bit非对称量化精度，激活值为8bit非对称量化精度。（RK2118不支持） • w4a16：权重为4bit非对称量化精度，激活值为16bit浮点精度。（仅RK3576/RV1126B支持） • w8a16：权重为8bit非对称量化精度，激活值为16bit浮点精度。（仅RK3562支持） • w4a8：权重为4bit非对称量化精度，激活值为8bit非对称量化精度。（暂不支持） • w16a16i：权重为16bit非对称量化精度，激活值为16bit非对称量化精度。（仅RV1103/RV1106支持） • w16a16i_dfp：权重为16bit动态定点量化精度，激活值为16bit动态定点量化精度。（仅RV1103/RV1106支持）
quantized_algorithm	计算每一层的量化参数时采用的量化算法，目前支持的量化算法有：normal，mmse，kl_divergence及gdq。默认值为normal。 normal量化算法的特点是速度较快，推荐量化数据集量一般为20-100张左右，更多的数据集量下精度未必会有进一步提升。 mmse量化算法由于采用暴力迭代的方式，速度较慢，但通常会比normal具有更高的精度，推荐量化数据集量一般为20-50张左右，用户也可以根据量化时间长短对量化数据集量进行适当增减。 kl_divergence量化算法所用时间会比normal多一些，但比mmse会少很多，在某些场景下 （feature分布不均匀时）可以得到较好的改善效果，推荐量化数据集量一般为20-100张左右。 gdq量化算法只在w4a16和w8a16下有效，能有效的提高w4a16和w8a16的权重量化精度，推荐量化数据集量为200张以上。
quantized_method	目前支持layer，channel或者group{SIZE}。默认值为channel。 • layer：每层的weight只有一套量化参数； • channel：每层的weight的每个输出通道都有一套量化参数，通常情况下channel会比layer精度更高。

	group{SIZE}：在channel的基础上，每个输出通道的weight在输入通道上又按{SIZE}细分为多个小组，每个小组有各自的一套量化参数，通常情况下group{SIZE}会比channel精度更高。{SIZE}是32到256之间的32的倍数值，如group32或group128。目前group{SIZE}只在 quantized_dtype为w4a16下有效。
float_dtype	用于指定非量化情况下的浮点的数据类型，目前支持的数据类型有float16。默认值为float16。
optimization_level	模型优化等级。默认值为3。 通过修改模型优化等级，可以关掉部分或全部模型转换过程中使用到的优化规则。该参数的默认值为3，打开所有优化选项。值为2或1时关闭一部分可能会对部分模型精度产生影响的优化选项，值为0时关闭所有优化选项。
target_platform	指定RKNN模型是基于哪个目标芯片平台生成的。目前支持“rv1103”、“rv1103b”、“rv1106”“rv1106b”、“rv1126b”、“rk2118”、“rk3562”、“rk3566”、“rk3568”、“rk3576”和“rk3588”。该参数对大小写不敏感。默认值为None。
custom_string	添加自定义字符串信息到RKNN模型，可以在runtime时通过query查询到该信息，方便部署时根据不同的RKNN模型做特殊的处理。默认值为None。
remove_weight	去除conv等权重以生成一个RKNN的从模型，该从模型可以与带完整权重的RKNN模型共享权重以减少内存消耗。默认值为False。
compress_weight	压缩模型权重，可以减小RKNN模型的大小。默认值为False。
single_core_mode	是否仅生成单核模型，可以减小RKNN模型的大小和内存消耗。默认值为False。目前仅对RK3588 / RK3576生效。默认值为False。
model_pruning	对模型进行无损剪枝。对于权重稀疏的模型，可以减小转换后RKNN模型的大小和计算量。默认值为False。
op_target	用于指定OP的具体执行目标（可以设置运行在NPU/CPU/GPU等），通过输出tensor名的方式，{'op0_output_name':'cpu','op1_output_name':'npu',...}，或者通过指定op_type的方式，{'op_type':'cpu'....}，同时还可以支持混合使用，{'op_type':'cpu ','op0_output_name':'cpu'....}。默认值为None。 其中，'op0_output_name'和'op1_output_name'为对应OP的输出tensor名，可以通过精度分析（accuracy_analysis）功能的返回结果中获取。'cpu'和'npu'则表示该tensor对应的OP的执行目标是CPU或NPU，目前可选的选项有：'cpu' / 'npu' / 'gpu' / 'auto'，其中，'auto'是自动选择执行目标。指定op_type的方式，同样需要通过精度分析返回结果中获取，可以将模型内所有该 op_type的执行目标设置到CPU或NPU，例如，{'Add':'cpu','Expand':'cpu',....}。
dynamic_input	用于根据用户指定的多组输入shape，来模拟动态输入的功能。格式为[[input0_shapeA, input1_shapeA, ...], [input0_shapeB, input1_shapeB, ...], ...]。

	默认值为None，实验性功能。 假设原始模型只有一个输入，shape为[1,3,224,224]，或者原始模型的输入shape本身就是动态的，如shape为[1,3,height,width]或[1,3,-1,-1]，但部署的时候，需要该模型支持3种不同的输入shape，如[1,3,224,224], [1,3,192,192]和[1,3,160,160]，此时可以设置dynamic_input= [[[1,3,224,224]], [[1,3,192,192]], [[1,3,160,160]]]，转换成RKNN模型后进行推理时，需传入对应shape的输入数据。 注： 1. 需要原始模型本身支持动态输入才可开启此功能，否则会报错。 2. 如果原始模型输入shape本身就是动态的，则只有动态的轴可以设置不同的值。
quantize_weight	在build接口的do_quantization为False情况下, 通过对一些权重进行量化以减小rknn模型的大小。 默认值为False。
remove_reshape	删除模型的输入和输出中可能存在的Reshape的OP，以提高模型运行时性能（因为目前很多平台的Reshape是跑在cpu上，相对较慢）。 默认为 False。 注：开启后可能会修改模型的输入或输出节点的shape，需要留意观察转换过程中的warning 打印，并在部署时也需要考虑输入和输出shape变化的影响。
sparse_infer	在已经稀疏化过的模型上进行稀疏化推理，以提高推理性能。目前仅对RK3576/RV1126B生效。默认为 False。
enable_flash_attention	是否启用Flash Attention。默认为 False。 注：FlashAttention是基于 https://arxiv.org/abs/2307.08691 实现, 通过高速缓存内循环实现加速 以及减少宽带使用，但是会导致模型增大，请根据具体场景和模型选择是否开启使用。更多 详情请查看“RKNN Compiler Support Operator List”的exSDPAttention说明。
auto_hybrid_cos_thresh	模型量化时开启自动混合量化中余弦距离的阈值。默认为 0.98。
auto_hybrid_euc_thresh	模型量化时开启自动混合量化中欧式距离的阈值。默认为 None，即默认不启用。
返回值	无

举例如下：

代码块

```
1 # model config
2 rknn.config(mean_values=[[103.94, 116.78, 123.68]],
3             std_values=[[58.82, 58.82, 58.82]],
4             quant_img_RGB2BGR=True,
5             target_platform='rk3566')
```

2.3 模型加载接口

RKNN-Toolkit2目前支持Caffe、TensorFlow、TensorFlow Lite、ONNX、DarkNet、PyTorch等模型的加载转换，这些模型在加载时需调用对应的接口，以下为这些接口的详细说明。

2.3.1 Caffe模型加载接口

API	load_caffe
描述	加载Caffe模型。（ARM64版本暂不支持该接口）
参数	model: Caffe模型文件（.prototxt后缀文件）路径。
	blobs: Caffe模型的二进制数据文件（.caffemodel后缀文件）路径。
	input_name: Caffe模型存在多输入时，可以通过该参数指定输入层名的顺序，形如['input1','input2','input3']，注意名字需要与模型输入名一致；默认值为None，表示按Caffe模型文件（.prototxt后缀文件）自动给定。
返回值	0: 导入成功。
	-1: 导入失败。

举例如下：

代码块

```
1 # 从当前路径加载mobilenet_v2模型
2 ret = rknn.load_caffe(model='./mobilenet_v2.prototxt',
3                       blobs='./mobilenet_v2.caffemodel')
```

2.3.2 TensorFlow模型加载接口

API	load_tensorflow
描述	加载TensorFlow模型。（ARM64版本暂不支持该接口）
参数	tf_pb : TensorFlow模型文件（.pb后缀）路径。
	inputs : 模型的输入节点（tensor名），支持多个输入节点。所有输入节点名放在一个列表中。
	input_size_list : 每个输入节点对应的shape，所有输入shape放在一个列表中。如示例中的ssd_mobilenet_v1模型，其输入节点对应的输入shape是[[1, 300, 300, 3]]。
	outputs : 模型的输出节点（tensor名），支持多个输出节点。所有输出节点名放在一个列表中。
	input_is_nchw : 模型的输入的layout是否已经是NCHW。默认值为False，表示默认输入layout为NHWC。
返回值	0: 导入成功。
	-1: 导入失败。

举例如下：

代码块

```
1  # 从当前目录加载ssd_mobilenet_v1_coco_2017_11_17模型
2  ret = rknn.load_tensorflow(tf_pb='./ssd_mobilenet_v1_coco_2017_11_17.pb',
3                               inputs=['Preprocessor/sub'],
4                               outputs=['concat', 'concat_1'],
5                               input_size_list=[[300, 300, 3]])
```

2.3.3 TensorFlow Lite模型加载接口

API	load_tflite
描述	加载TensorFlow Lite模型。（ARM64版本暂不支持该接口）
参数	model: TensorFlow Lite模型文件（.tflite后缀）路径。
	input_is_nchw: 模型的输入的layout是否已经是NCHW。默认值为False，即默认输入layout为NHWC。
返回值	0：导入成功。
	-1：导入失败。

举例如下：

代码块

```
1 # 从当前目录加载mobilenet_v1模型
2 ret = rknn.load_tflite(model='./mobilenet_v1.tflite')
```

2.3.4 ONNX模型加载

API	load_onnx
描述	加载ONNX模型。
参数	model: ONNX模型文件（.onnx后缀）路径。
	inputs: 模型输入节点（tensor名），支持多个输入节点，所有输入节点名放在一个列表中。默认值为None，表示从模型里获取。
	input_size_list: 每个输入节点对应的shape，所有输入shape放在一个列表中。如inputs有设置，则input_size_list也需要被设置。默认值为None。
	input_initial_val: 设置模型输入的初始值，格式为ndarray的列表。默认值为None。主要用于将某些输入固化为常量，对于不需要固化为常量的输入可以设为None，如[None, np.array([1])]。
	outputs: 模型的输出节点（tensor名），支持多个输出节点，所有输出节点名放在一个列表中。默认值为None，表示从模型里获取。
返回值	0：导入成功。
	-1：导入失败。

举例如下：

代码块

```
1  # 从当前目录加载arcface模型
2  ret = rknn.load_onnx(model='./arcface.onnx')
```

2.3.5 DarkNet模型加载接口

API	load_darknet
描述	加载DarkNet模型。（ARM64版本暂不支持该接口）
参数	model: DarkNet模型文件（.cfg后缀）路径。
	weight: 权重文件（.weights后缀）路径。
返回值	0：导入成功。
	-1：导入失败。

举例如下：

代码块

```
1  # 从当前目录加载yolov3-tiny模型
2  ret = rknn.load_darknet(model='./yolov3-tiny.cfg', weight='./yolov3.weights')
```

2.3.6 PyTorch模型加载接口

API	load_pytorch
描述	加载PyTorch模型。支持量化感知训练（QAT）模型，但需要将torch版本更新至1.9.0以上。
参数	model: PyTorch模型文件（.pt后缀）路径，而且需要是torchscript格式的模型。
	input_size_list: 每个输入节点对应的shape，所有输入shape放在一个列表中。
返回值	0：导入成功。
	-1：导入失败。

举例如下：

代码块

```
1  # 从当前目录加载resnet18模型
```

```
2 ret = rknn.load_pytorch(model='./resnet18.pt', input_size_list=[[1,3,224,224]])
```

2.4 构建RKNN模型

用户加载原始模型后，下一步就是通过 `rknn.build()` 接口构建RKNN模型。构建模型时，用户可以选择是否

进行量化，量化可以减小模型的大小和提高在RKNPU上的性能。

参数	描述
do_quantization	是否对模型进行量化。默认值为True。
dataset	用于量化校正的数据集。目前支持文本文件格式，用户可以把用于校正的图片（jpg或png格式）或numpy文件路径放到一个.txt文件中。文本文件里每一行一条路径信息。 如： a.jpg b.jpg 或 a.npy b.npy 如有多个输入，则每个输入对应的文件用空格隔开，如： a.jpg a2.jpg b.jpg b2.jpg 或 a.npy a2.npy b.npy b2.npy 注：量化图片建议选择与预测场景较吻合的图片。
rknn_batch_size	模型的输入Batch参数调整。默认值为None，表示不进行调整。如果大于1，则可以在一次推理中同时推理多帧输入图像或输入数据，如MobileNet模型的原始input维度为[1, 224, 224, 3]，output维度为[1, 1001]，当rknn_batch_size设为4时，input的维度变为[4,224, 224, 3]，output维度变为[4,1001]。 注： 1.rknn_batch_size只有在NPU多核的平台上可以提高性能（提升核心利用率），因rknn_batch_size的值建议与核心数匹配。 2.rknn_batch_size修改后，模型的input/output的shape都会被修改，使用inference推理模型时需要设置相应的input的大小，后处理时，也需要对返回的outputs进行处理。

auto_hybrid

是否开启自动混合量化调整精度或溢出。默认值为False，表示不进行调整。当模型经行量化时，开启自动混合量化会将低于指定阈值的余弦距离/欧式距离的op转为fp16计算(目前仅在w8a8量化下支持);当模型未进行量化时，开启自动混合量化会将超出fp16数值范围的op转为int16计算。

注：

1.余弦距离和欧式距离的阈值可通过config接口中的 auto_hybrid_cos_thresh和 auto_hybrid_euc_thresh设置。

举例如下：

代码块

```
1 # 构建RKNN模型，并且做量化
2 ret = rknn.build(do_quantization=True, dataset='./dataset.txt')
```

2.5 导出RKNN模型

用户通过 rknn.build() 接口构建了 RKNN 模型后，可以通过 rknn.export_rknn() 接口导出RKNN模型文件（.rknn 后缀），以便后续模型的部署。

API	export_rknn
描述	将RKNN模型保存到指定文件中（.rknn后缀）。
参数	export_path: 导出模型文件的路径。
返回值	0：导出成功。
	-1：导出失败。

举例如下：

代码块

```
1 # 将构建好的RKNN模型保存到当前路径的mobilenet_v1.rknn文件中
2 ret = rknn.export_rknn(export_path='./mobilenet_v1.rknn')
```

3. 模型转换代码测试

01_convert_rknn_model

代码块

```

1  from rknn.api import RKNN
2
3  if __name__ == '__main__':
4      rknn = RKNN(verbose=True, verbose_file="log.txt")
5
6      # 调用config接口配置要生成的RKNN模型
7      rknn.config(
8          mean_values=[[123.675, 116.28, 103.53]], # mean_values表示预处理要减去的
          # 均值化参数
9          std_values=[[58.395, 58.395, 58.395]], # std_values 表示预处理要除的标准
          # 化参数
10         quantized_dtype="asymmetric_quantized-8", # quantized_dtype表示量化类型
11         quantized_algorithm='normal', # quantized_algorithm表示量化的算法
12         quantized_method='channel', # quantized_method 表示量化的方式
13         quant_img_RGB2BGR=False, # 表示在加载量化图像时是否需要先做RGB2BGR的操作
14         target_platform="rk3588", # target_platform表示RKNN模型的运行平台
15         float_dtype="float16", # float_dtype表示RKNNM中的默认浮点数类型
16         optimization_level=3, # optimization_level表示模型优化等级
17         custom_string="this is my rknn model ", # custom_string表示添加的自定义
          # 字符串信息
18         remove_weight=False, # remove_weight表示生成一个去除权重信息的从模型
19         compress_weight=False, # compress_weight表示压缩模型权重，可以减小模型体积
20         inputs_yuv_fmt=None, # 表示RKNN模型输入数据的YUV格式
21         single_core_mode=False, #表示构建的RKNN模型运行在单核心模式，只适用于RK3588
22     )
23     # 添加load_xxx接口进行常用深度学习模型的导入
24     rknn.load_pytorch(
25         model="./resnet18.pt", # model表示加载模型的地址
26         input_size_list=[[1, 3, 224, 224]], # input_size_list表示模型输入节点对应
          # 图片的尺寸和通道数
27     )
28
29     # 使用build接口来构建RKNN模型
30     rknn.build(
31         do_quantization=True, # do_quantization 表示是否对RKNN模型进行量化操作，
32         dataset="dataset.txt", # dataset 表示要量化的图片
33     )
34
35     # 调用export_rknn接口导出RKNN模型
36     rknn.export_rknn(
37         export_path="resnet18.rknn" # export_path表示到处的RKNN模型路径
38     )
39     rknn.release()

```

执行程序：

输出:

```

topeet@ubuntu: ~/rknn/04_code/01_convert_rknn_model
D RKNN: [04:28:20.606] 13 Conv bias.28_38_bias INT32 (256) | 0x00b31a00 0x00b32200 0x00000800
D RKNN: [04:28:20.606] 14 ConvAddRelu weight.60 INT8 (256,128,1,1) | 0x0099f000 0x009a7000 0x00008000
D RKNN: [04:28:20.606] 14 ConvAddRelu bias.30_40_bias INT32 (256) | 0x00b32200 0x00b32a00 0x00008000
D RKNN: [04:28:20.606] 15 ConvRelu weight.64 INT8 (256,256,3,3) | 0x0090f000 0x0099f000 0x00090000
D RKNN: [04:28:20.606] 15 ConvRelu bias.32_44_bias INT32 (256) | 0x00b32a00 0x00b33200 0x00008000
D RKNN: [04:28:20.606] 16 ConvAddRelu weight.68 INT8 (256,256,3,3) | 0x0087f000 0x0090f000 0x00090000
D RKNN: [04:28:20.606] 16 ConvAddRelu bias.34_47_bias INT32 (256) | 0x00b33200 0x00b33a00 0x00008000
D RKNN: [04:28:20.606] 17 ConvRelu weight.8 INT8 (512,256,3,3) | 0x0075f000 0x0087f000 0x00120000
D RKNN: [04:28:20.606] 17 ConvRelu bias.3_51_bias INT32 (512) | 0x00b33a00 0x00b34a00 0x00001000
D RKNN: [04:28:20.606] 18 Conv weight.7 INT8 (512,512,3,3) | 0x0051f000 0x0075f000 0x00240000
D RKNN: [04:28:20.606] 18 Conv bias.4_54_bias INT32 (512) | 0x00b34a00 0x00b35a00 0x00001000
D RKNN: [04:28:20.606] 19 ConvAddRelu weight.3 INT8 (512,256,1,1) | 0x004ff000 0x0051f000 0x00020000
D RKNN: [04:28:20.606] 19 ConvAddRelu bias.5_56_bias INT32 (512) | 0x00b35a00 0x00b36a00 0x00001000
D RKNN: [04:28:20.606] 20 ConvRelu weight.2 INT8 (512,512,3,3) | 0x002bf000 0x004ff000 0x00240000
D RKNN: [04:28:20.606] 20 ConvRelu bias.2_60_bias INT32 (512) | 0x00b36a00 0x00b37a00 0x00001000
D RKNN: [04:28:20.606] 21 ConvAddRelu weight.6 INT8 (512,512,3,3) | 0x0007f000 0x002bf000 0x00240000
D RKNN: [04:28:20.606] 21 ConvAddRelu bias.36_63_bias INT32 (512) | 0x00b37a00 0x00b38a00 0x00001000
D RKNN: [04:28:20.606] 22 Conv x.1_GlobalAveragePool_2conv0_i1 INT8 (1,512,7,7) | 0x00b38a00 0x00b3ec00 0x00006200
D RKNN: [04:28:20.606] 22 Conv x.1_GlobalAveragePool_2conv0_i2 INT32 (512) | 0x00b3ec00 0x00b3fc00 0x00001000
D RKNN: [04:28:20.606] 23 Conv weight.1 INT8 (1000,512,1,1) | 0x00002000 0x0007f000 0x0007d000
D RKNN: [04:28:20.606] 23 Conv bias.1 INT32 (1000) | 0x00000000 0x00002000 0x00002000
D RKNN: [04:28:20.606] 24 Reshape 572_mm_tp_rs_i1 INT64 (2) | 0x00b3fc00 0x00b3fc10 0x00000010
D RKNN: [04:28:20.606] -----+-----
D RKNN: [04:28:20.606] Total Internal Memory Size: 1127KB
D RKNN: [04:28:20.606] Total Weight Memory Size: 11521.1KB
D RKNN: [04:28:20.606] -----
D RKNN: [04:28:20.606] <<<<<<< end: rknn::RKNNMemStatisticsPass
I rknn building done.
(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/01_convert_rknn_model$ ls
check0_base_optimize.onnx  convert_rknn_model.py  resnet18.pt
check2_correct_ops.onnx    dataset.txt             resnet18.rknn
check3_fuse_ops.onnx       log.txt                 space_shuttle_224.jpg
(rknn-toolkit2) topeet@ubuntu:~/rknn/04_code/01_convert_rknn_model$

```